

معماری نرم افزار

پروژه: سیستم مدیریت شهریه مجتمع آموزشی

1. مقدمه

در اولین مرحله از طراحی فنی این پروژه، باید معماری نرم افزار به صورت دقیق مشخص شود؛ چون تمام تصمیم های بعدی مانند طراحی دیتابیس، ساخت API، پیاده سازی رابط کاربری، اتصال به درگاه بانکی و کنترل منطق مالی، به این معماری وابسته هستند.

پروژه حاضر یک سیستم ساده ثبت اطلاعات نیست. این سامانه با داده های مالی، تراکنش های پرداخت، مدیریت بدهی، ثبت اقساط، کنترل چک، تخفیف، گزارش گیری و ارسال اعلان سروکار دارد. بنابراین معماری آن باید سه ویژگی اصلی را تامین کند:

- جداسازی منطق مالی از رابط کاربری
- قابلیت توسعه و نگهداری در آینده
- امنیت و صحت در عملیات مالی

در این معماری، سیستم به چند لایه مستقل تقسیم می شود تا هر بخش فقط مسئول وظایف مشخص خود باشد و وابستگی ها به درستی کنترل شوند. اصل اصلی این معماری این است که:

منطق اصلی کسب و کار نباید به دیتابیس، رابط کاربری، درگاه بانکی یا فناوری های جانبی وابسته باشد. این اصل در یک سیستم مالی بسیار مهم است، چون قوانین مالی باید ثابت، قابل اعتماد و قابل تست باشند.

تعدد فرآیندها

طبق DFD و Backlog، سیستم فقط یک فرآیند ندارد؛ بلکه شامل چندین سناریوی مستقل و مرتبط است:

- تعریف پایه تحصیلی
- ثبت دانش آموز
- تعیین شهریه
- اعمال تخفیف
- ثبت چک و اقساط
- پرداخت حضوری
- پرداخت آنلاین
- تایید تراکنش

- صدور رسید
- گزارش بدهکاران
- ارسال پیامک یادآوری

این حجم از فرآیندها نیاز به معماری لایه‌ای و قابل مدیریت دارد.

نیاز به توسعه در آینده

در پروژه‌های واقعی، معمولاً در آینده نیازهایی مثل این‌ها اضافه می‌شود:

- گزارش‌های مالی جدید
- اتصال به چند درگاه بانکی
- پنل والدین
- پنل حسابداری
- ثبت تخفیف‌های ترکیبی
- امکان پرداخت بخشی
- فایل خروجی Excel یا PDF

ساختار کلی معماری

معماری سیستم به چهار لایه اصلی تقسیم می‌شود:

- `Presentation Layer`
- `Application Layer`
- `Domain Layer`
- `Infrastructure Layer`

وابستگی لایه‌ها باید به سمت داخل باشد؛ یعنی لایه‌های بیرونی می‌توانند به لایه‌های داخلی وابسته باشند، ولی برعکس آن مجاز نیست.

ترتیب وابستگی به این صورت است:

`Presentation -> Application -> Domain`

و لایه `Infrastructure` پیاده‌ساز نیازهای لایه‌های داخلی است.

لایه Domain

لایه Domain مهم‌ترین بخش سیستم است و هسته اصلی نرم‌افزار را تشکیل می‌دهد. تمام موجودیت‌های اصلی پروژه و قوانین پایه کسب‌وکار در این لایه قرار می‌گیرند.

این لایه نباید به هیچ تکنولوژی خاصی وابسته باشد. یعنی نه به دیتابیس، نه به فریم‌ورک وب، نه به درگاه بانکی و نه به سرویس پیامک.

مسئولیت‌ها

- تعریف موجودیت‌های اصلی سیستم
- تعریف قوانین پایه مالی
- تعریف ارتباط مفهومی بین داده‌ها
- جلوگیری از وضعیت‌های نامعتبر

موجودیت‌های اصلی بر اساس نمودارها

طبق Class Diagram و ERD، موجودیت‌های اصلی این پروژه عبارت‌اند از:

- «دانشجو»
- «شهریه»
- «تخفیف»
- «پرداخت»
- «قسط»
- «چک»
- «رسید»
- «گزارش بدهکار»

نمونه مسئولیت هر موجودیت

- «دانشجو»

- نگهداری اطلاعات پایه تحصیلی

- «شهریه»

- مبلغ شهریه مصوب برای پایه و سال تحصیلی

- «تخفیف»

- تخفیف درصدی یا مبلغی

- «پرداخت»

- ثبت عملیات پرداخت با نوع و مبلغ و زمان

- «قسط»

- مدیریت پرداخت‌های قسطی

- «چک»

- اطلاعات چک، سررسید و وضعیت وصول

- «رسید»

- اطلاعات رسید پرداخت

قوانین پایه در Domain

نمونه قوانین مهمی که باید در این لایه قرار بگیرند:

- مبلغ شهریه نمی‌تواند منفی باشد.
- مبلغ پرداخت نمی‌تواند صفر یا منفی باشد.
- تخفیف نباید از کل شهریه بیشتر شود.
- مجموع اقساط نباید از مبلغ قابل پرداخت بیشتر شود.
- برای یک تراکنش موفق، فقط یک ثبت نهایی پرداخت انجام شود.
- وضعیت چک باید مشخص باشد: در انتظار، وصول شده، برگشتی.

- بدهی دانش‌آموز برابر است با:

بدهی = شهریه - تخفیف - مجموع پرداخت‌های تاییدشده

لایه Application

این لایه مسئول پیاده‌سازی سناریوهای واقعی سیستم است. هر چیزی که در Product Backlog، DFD و Sequence Diagram به‌عنوان فرآیند یا عملیات آمده، در این لایه پیاده‌سازی می‌شود. Domain می‌گوید قوانین چیست؛ Application می‌گوید این قوانین در چه ترتیبی اجرا شوند.

مسئولیت‌ها

- اجرای Use Case ها
- هماهنگ‌سازی بین Domain و Infrastructure
- اعتبارسنجی سطح کاربردی
- مدیریت جریان عملیات
- کنترل تراکنش‌ها

Use Case های اصلی پروژه

بر اساس مستندات، Use Case های اصلی عبارت‌اند از:

- «ثبت نام دانشجو»
- «تعریف: آموزش»
- «تعریف تخفیف»
- «تخصیص تخفیف به دانشجو»
- «طرح اقساطی ثبت نام»
- «چک ثبت نام»
- «ثبت در پرداخت شخصی»
- «ایجاد درخواست پرداخت آنلاین»

- «تأیید تراکنش بانکی»
- «رسید دیجیتال نسل»
- «گزارش بدهکاران ماهانه تولید کنید»
- «ارسال بدهی یادآوری»

مثال از منطق Application

برای فرآیند پرداخت آنلاین، Application باید این مراحل را مدیریت کند:

1. دریافت درخواست پرداخت از کاربر
2. استخراج بدهی دانش آموز
3. اعتبارسنجی مبلغ پرداخت
4. ایجاد درخواست به درگاه بانکی
5. ثبت تراکنش اولیه
6. دریافت پاسخ بانک
7. تأیید یا رد تراکنش
8. ثبت پرداخت نهایی
9. صدور رسید
10. به روزرسانی بدهی دانش آموز

لایه Infrastructure

این لایه مسئول پیاده‌سازی جزئیات فنی است. هر چیزی که به تکنولوژی وابسته باشد در این بخش قرار می‌گیرد.

مسئولیت‌ها

- اتصال به پایگاه داده
- پیاده‌سازی Repositoryها
- ارتباط با درگاه بانکی
- ارسال پیامک
- ذخیره‌سازی فایل رسید
- لاگ‌گیری و مانیتورینگ
- تنظیمات امنیتی و دسترسی

اجزای اصلی Infrastructure

- «پایداری»
- جداول پایگاه داده
- ORM یا لایه پرس و جو
- پیاده‌سازی مخزن
- «یکپارچه سازی دروازه بانکی»
- اتصال به API بانک
- ثبت توکن، RefId، وضعیت تراکنش
- «ادغام ارائه دهنده پیامک‌ها»
- ارسال پیامک یادآوری
- «ذخیره سازی فایل»
- ذخیره رسیدها یا فایل‌های PDF
- «login»
- ثبت خطاها و رویدادهای حساس مالی

نقش این لایه در پروژه شما

طبق نمودارها، این لایه باید موارد زیر را اجرا کند:

- ذخیره اطلاعات دانش آموز و شهریه
- ثبت تراکنش های پرداخت
- نگهداری اطلاعات اقساط و چک
- ثبت خروجی رسید
- نگهداری گزارش بدهکاران
- ارسال پیامک یادآوری
- ارتباط با درگاه بانکی در فرآیند پرداخت آنلاین

نکته مهم

Infrastructure فقط باید پیاده ساز قراردادها باشد، نه محل تصمیم گیری اصلی. مثلاً محاسبه بدهی نباید در SQL یا درگاه بانکی انجام شود؛ این منطق متعلق به Domain/Application است.

لایه Presentation

این لایه وظیفه تعامل با کاربران را برعهده دارد. کاربران سیستم طبق DFD عبارت اند از:

- `مدیر سیستم`
- `مسئول مالی`
- `والدین / دانش آموز`

مسئولیت ها

- دریافت ورودی از کاربر
- نمایش فرم ها و گزارش ها
- ارسال درخواست به Application
- نمایش نتیجه عملیات
- مدیریت احراز هویت و سطح دسترسی در سطح UI/API

بخش‌های پیشنهادی رابط کاربری

برای این پروژه، Presentation می‌تواند شامل بخش‌های زیر باشد:

- پنل مدیریت پایه‌ها و شهریه
- فرم ثبت دانش‌آموز
- صفحه ثبت تخفیف
- پنل ثبت پرداخت حضوری
- پنل ثبت اقساط و چک
- صفحه پرداخت آنلاین
- صفحه نمایش رسید
- گزارش بدهکاران
- صفحه ارسال یا مشاهده پیامک‌های یادآوری

Presentation فقط باید داده را بگیرد و به Use Case مناسب ارسال کند. این لایه نباید:

تطبیق معماری با نمودارهای پروژه

کلاس‌های اصلی مثل دانش‌آموز، شهریه، پرداخت، تخفیف، چک و اقساط در لایه Domain قرار می‌گیرند. کلاس‌های وابسته به تعاملات مانند درگاه بانکی و پیامک در لایه Infrastructure پیاده‌سازی می‌شوند.

فرآیندهای دیده‌شده در DFD سطح 1، مستقیماً به Use Case‌های Application تبدیل می‌شوند. برای مثال:

- تعریف مبلغ شهریه ← تعریف مورد استفاده شهریه
- پرداخت آنلاین ← ایجاد مورد استفاده درخواست پرداخت آنلاین
- تأیید تراکنش پرداخت ← مورد تأیید تراکنش بانکی

- صدور رسید دیجیتال ← مورد استفاده تولید رسید
- ارسال پیامک یادآوری ← مورد استفاده یادآوری ارسال پیامک

تطبیق با Sequence Diagram

نمودار توالی پرداخت آنلاین نشان می‌دهد که عملیات باید از UI شروع شود، از طریق Application کنترل شود، به Infrastructure برای ارتباط با بانک برسد، نتیجه را ثبت کند و سپس رسید تولید شود. این دقیقاً با Clean Architecture منطبق است.

تطبیق با ERD

موجودیت‌های ERD پایه طراحی دیتابیس در Infrastructure هستند، ولی منطق استفاده از آن‌ها در Domain و Application قرار می‌گیرد.

سناریو: پرداخت آنلاین شهریه

بر اساس Sequence Diagram، اجرای این سناریو در معماری به این شکل خواهد بود:

1. کاربر در UI درخواست پرداخت را ثبت می‌کند.
2. Presentation درخواست را به `خدمات پرداخت` در Application ارسال می‌کند.
3. Application بدهی دانش‌آموز را از مخزن‌ها می‌خواند.
4. Domain اعتبار مبلغ و وضعیت پرونده مالی را بررسی می‌کند.
5. Application یک تراکنش اولیه ایجاد می‌کند.
6. Infrastructure درخواست را به درگاه بانکی ارسال می‌کند.
7. بانک نتیجه تراکنش را برمی‌گرداند.
8. Application نتیجه را تحلیل می‌کند.
9. اگر موفق بود:

پرداخت نهایی ثبت می‌شود

بدهی به‌روزرسانی می‌شود

رسید ایجاد می‌شود

10. اگر ناموفق بود:

وضعیت تراکنش ناموفق ثبت می‌شود

پیام خطا به کاربر نمایش داده می‌شود

این تفکیک باعث می‌شود هیچ بخشی از سیستم بیش از حد مسئولیت نداشته باشد.

مزایای این معماری برای پروژه

نگهداری آسان

اگر در آینده نیاز باشد UI تغییر کند، منطق اصلی سیستم آسیب نمی‌بیند.

می‌توان بدون بازنویسی هسته سیستم، امکانات جدید اضافه کرد:

- چند درگاه بانکی
- چند نوع تخفیف
- گزارش‌های مالی پیچیده‌تر
- اپلیکیشن موبایل

چون دسترسی مستقیم UI به دیتابیس و منطق مالی وجود ندارد، احتمال دستکاری یا خطای ساختاری پایین‌تر می‌آید.

اگر بانک یا سرویس پیامکی عوض شود، فقط پیاده‌سازی Infrastructure تغییر می‌کند.

محدودیت‌ها و ملاحظات

- نباید منطق مالی در Controller یا Form نوشته شود.
 - نباید Queryهای دیتابیس تبدیل به محل تصمیم‌گیری کسب‌وکار شوند.
 - باید مسئولیت هر لایه کاملاً مشخص بماند.
 - موجودیت‌های Domain نباید به کلاس‌های دیتابیزی یا فریم‌ورک وابسته شوند.
 - Use Caseها باید واضح، مستقل و قابل تست نوشته شوند.
- اگر این اصول رعایت نشوند، معماری فقط در ظاهر لایه‌ای خواهد بود و در عمل بی‌اثر می‌شود.

جمع‌بندی نهایی

این انتخاب مستقیماً با مستندات پروژه شما هماهنگ است، چون:

- در نمودار کلاس، موجودیت‌های مالی و آموزشی مشخص‌اند
- در DFD، فرآیندهای عملیاتی روشن هستند
- در Sequence Diagram، جریان پرداخت آنلاین و تعامل با بانک مشخص شده
- در ERD، ساختار داده‌ها تعریف شده است

بنابراین این معماری می‌تواند:

- منطق مالی را متمرکز و امن نگه دارد
- فرآیندهای پرداخت را قابل کنترل کند
- توسعه آینده سیستم را ساده‌تر کند
- تست و نگهداری پروژه را ممکن سازد

خروجی این مرحله

خروجی رسمی مرحله «معماری نرم‌افزار» برای این پروژه به این صورت است:

- لایه‌ها: `Presentation` , `Application` , `Domain` , `Infrastructure`

- هسته دامنه: دانش آموز، شهریه، تخفیف، پرداخت، اقساط، چک، رسید
- سناریوهای اصلی Application: ثبت نام، تعریف شهریه، ثبت پرداخت، پرداخت آنلاین، تایید تراکنش، گزارش بدهکاران، ارسال پیامک
- زیرساخت‌ها: دیتابیس، درگاه بانکی، سرویس پیامک، ذخیره رسید